

Tic-Tac-Toe

Final Report for the Distributed Systems Course [2024-2025]

Filippo Pracucci

`filippo.pracucci@studio.unibo.it`

September 23, 2026

This report describes the project developed for the 2024-2025 Distributed Systems course called **Tic-Tac-Toe**. The project consists on a distributed implementation of a variant of the classic multiplayer game Tic-Tac-Toe, in which two players can play against each other, creating a game or joining an existing one. The game consists in the two players placing alternatively their marks on a 3x3 grid; the first player who succeeds in placing three marks in an horizontal, vertical, or diagonal row wins the game. The variant consists in removing a random mark from the grid at each turn, after the third mark of the player has been placed, meaning that every player can have at most three marks on the grid at any time. In this way, there is no possibility of a draw, because the game can potentially last forever.

Disclaimer

During the preparation of this work, the author used GitHub Copilot to assist with code completion and an agent configured in the file `.github/agents/docs.agent.md` to generate the API documentation.

After using this tools/services, the author reviewed and edited the content as needed and takes full responsibility for the content of the final report/artifact.

1 Concept

The **Tic-Tac-Toe** game can be played in two modes:

- *local*: in which two players can play against each other on the same machine;
- *remote*: in which two players can play against each other on different machines, connected to the same network.

In case of *local* mode, the game starts immediately, so the GUI with the grid is displayed; while in case of *remote* mode the game starts with a screen representing a

lobby menu, in which the players can choose to either create a new game or join an existing one. After that the game starts and the GUI with the grid is displayed. During the game, there is the possibility to chat with the other player through the terminal used to launch the game. From now on, the report will focus on the *remote* mode, since it is the one addressing the distributed system features.

1.1 Use case description

The users interact with the system through a computer connected to the network. During the game, the players do not interact with the system very frequently, since it is a turn-based game, so they have to wait for the other player to make a move; however, they can chat with the other player, so in that case the interaction frequency gets higher. The game interactions are done by clicking on the GUI's grid with the mouse to place a mark, while the chat interactions are done by typing via the keyboard in the terminal.

The only data that needs to be stored is the list of the games that have been created, but not yet full, so that another player can join them. This is done by the server through a temporary file.

2 Requirements Elicitation and Analysis

2.1 Functional requirements

1. A player can create a new game or join an existing one while in the lobby menu;
2. a player can select which game to join from the list of available ones in the lobby menu and the only symbol available for the game selected;
3. a player can place a mark (*X* or *O*) on the grid during its turn via the GUI clicking with the mouse;
4. if a player tries to place a mark on a cell that is already occupied or when it is not its turn, the action is silently ignored;
5. a player can chat with the other player during the game via the terminal typing with the keyboard;
6. a player can see the current state of the game through the grid represented in the GUI;
7. a player can leave the game at any time pressing the *Esc* key, which will cause its defeat and the return of both players to the lobby menu;
8. the result of the game is displayed to both players' lobby menu at the end of the game;
9. after the third turn of a player, a random mark of that player is removed from the grid at each turn;

10. if the opponent disconnects from the game, the other player is notified and returned to the lobby menu;
11. if the server goes down, both players are notified and terminated;
12. the game ends when one of the two players places three marks in an horizontal, vertical, or diagonal row; so the players return to the lobby menu and the result of the game is displayed.

2.2 Non-functional requirements

1. The game must run on different operating systems, such as Windows, Linux and macOS;
2. the server must be able to handle multiple games being played simultaneously by different pairs of players;
3. latency must be low enough to allow for a smooth and responsive gaming experience, keeping the consistency of the game state between the two players.

2.3 Relevant Distributed System Features

- **Transparency:** the only distribution detail known to the users is that if they want to play against another player not on the same machine, they have to select the *remote* mode; all the other distribution aspects are hidden from the users, such as the architecture and the communications. Conversely, the development perspective is aware of the distribution, due to the explicit design and management of the system.
- **Fault tolerance and dependability:** failures client-side are handled by terminating the game and returning the other player to the lobby menu, causing the disconnected player's defeat. While failures server-side are handled by terminating the game and notifying both players of the server crash. It is not expected a recovery of the ongoing games after a server crash.
- **Scalability:** the server is able to handle multiple games simultaneously, but given that at the moment there is only one server, the scalability is limited by the server's resources. This means that in the future, it will be possible to increase the server replicas or resources in order to increase the scalability of the system.
- **Security:** the application does not handle sensitive data, so neither authentication nor authorization mechanisms are required. All the users have the same role, so also the same permissions, and they are considered trusted.
- **Resource sharing:** since the turn-based nature of the game, synchronization is a core aspect, and that is handled by means of overriding the state of the game in a client when it differs from the one owned by the server. So the server holds the

state of the game which is updated, and the clients are synchronized accordingly, by means of an override.

- **Openness, interoperability, heterogeneity:** the system is designed not to interact with external systems and it is not expected to operate with heterogeneous components.
- **Evolvability, maintainability:** the evolvability of the application is limited by the nature of the game; but the aspects that can be extended are the players management, maybe with an authentication system, in order to allow players to have a profile and a ranking. Other than that, the distributed aspects can be enhanced, for example by adding more servers to increase the scalability of the system and implementing a recovery mechanism.
- **Performance and concurrency:** since the game is turn-based, performances are not a concern, so latency and throughput are not critical aspects. The concurrency is limited to the number of games that can be played in parallel, which is limited by the server's resources.

3 Design

3.1 Architecture

The project follows the **MVC (Model-View-Controller)** architecture for the local application structure:

- the model is implemented in the `tic_tac_toe.model` package, which contains the domain logic (e.g. the game objects, the grid, etc.);
- the view is implemented in the `tic_tac_toe.view` package, which is responsible for the game interfaces and the lobby menu;
- the controller is implemented in the `tic_tac_toe.controller` package, which contains the definitions of input and event handlers, beyond their local implementations. In addition, `tic_tac_toe.remote` package contains all the aspects related to the distributed part of the application, such as the servers, the clients and the communications.

While the distributed part follows the **Event-Based Client-Server** architecture:

- a central server (`LobbyCoordinator`) acts as a coordinator managing the creation and joining of games. In addition, it handles the creation of the game server (`TicTacToeCoordinator`) instances and the connection of the clients to them;
- each user runs a client (`TicTacToeTerminal`) which connects initially to the central server to create or join a game, and after that it connects to the provided game server to play the game. During the game, the client communicates with the coordinator of the game it is playing, in order to update the game state;

- a *client-server* architecture is adopted to simplify the coordination by delegating the management of the game state to a single and centralized entity, while a *peer-to-peer* architecture would have required a more complex coordination mechanism to ensure the consistency of the game state between the clients;
- the game server publishes periodically **ControlEvent** events of type **TIME_ELAPSED** with the current game state and consumes user input events. While terminals publish user input events and consumes all types of **ControlEvent**.

3.2 Infrastructure

The system infrastructure is shown in fig. 1.

- **Infrastructural components:**
 - each player is a **client**, which runs a **TicTacToeTerminal**:
 - * it listens for user inputs and sends them to the game server;
 - * it receives the game state from the game server, so it updates the GUI accordingly;
 - the **central server** is a **LobbyCoordinator**:
 - * it listens for client requests to create or join a game;
 - * it manages the creation of game servers and sends the relative addresses to the clients;
 - * it keeps track of the list of games that have been created and are joinable by another player;
 - each game is managed by a **game server**, which is a **TicTacToeCoordinator**:
 - * it runs the game loop and updates the game state, without neither listening for inputs nor rendering the GUI;
 - * it receives the inputs from the clients;
 - * it sends periodically the game state to the clients, so they can update their GUI accordingly;
 - data is not stored persistently, so there is no need for a database or any other storage component.
- **Components distribution:**
 - the **LobbyCoordinator** is a single instance that runs on a single machine;
 - each **TicTacToeCoordinator** instance runs on the same machine, but as separate processes from the **LobbyCoordinator**, since they are created by it;
 - each **TicTacToeTerminal** can be distributed on different machines, as long as they are able to reach the **LobbyCoordinator**;

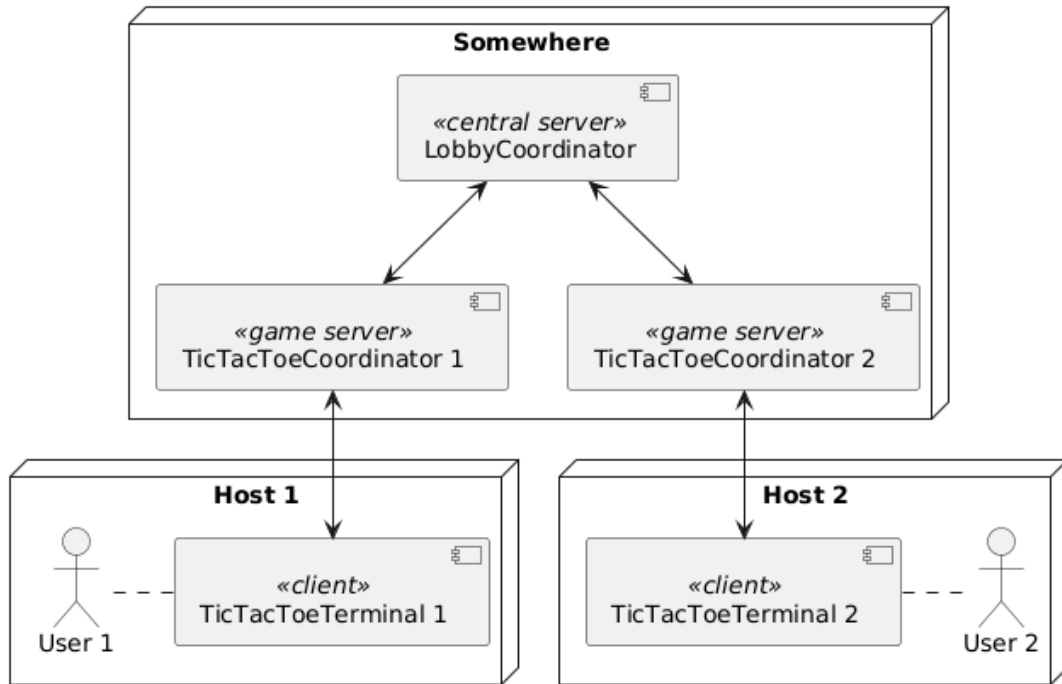


Figure 1: Component diagram

- however, for simplicity during development and testing, both clients and server are executed on the same machine.

- **Components discovery:**

- the `LobbyCoordinator` is reachable by the clients through an exposed IP address and port number;
- each client has to know the address of the `LobbyCoordinator` in order to connect to it. Then the clients are provided by the central server with the address of the `TicTacToeCoordinator` to connect to.

3.3 Modelling

3.3.1 Domain entities

The domain entities are shown in fig. 2 and the main ones are:

- **Player:** represents a player of a game, identified by a unique symbol in the game.
- **Symbol:** represents the symbol associated to a player, which can be either *X* or *O*, and will be used by the player to mark grid cells.
- **Mark:** represents the positioning of a symbol in a cell of the grid by a player.

- **Grid**: represents the board of the game, which by default is a 3x3 grid of cells.
- **Cell**: represents a cell of the grid, which can be either empty or occupied by a mark.
- **TicTacToe**: represents the global game state, including the grid, the list of players and marks placed, other than tracking the current turn.

The **Mark** class extends **GameObject**, because it represents the mark placed by a player in a cell of the grid as an object in the game view. Furthermore, **GameObject** other than being characterized by a position and a size in the view, it exposes also an **override** method, which is used for the moment only to override the marks of the game state.

The state of the game is represented by the **TicTacToe** class, which contains the references to all the relevant components, such as the grid, the list of players and the list of marks placed; in addition, it manages the current turn, which for simplicity is represented by the symbol of the active player. The game state can be overridden by means of the **override** method, in which every attribute is overridden by the corresponding one of the passed game state, only if the two differ.

The game state is kept both in the game server and in the clients, but to guarantee consistency, the game server has the one considered authoritative, so the clients override their game state with the one received periodically from the server by the **TIME_ELAPSED** event. While, the inputs are captured by the clients and the corresponding **InputEvent** is sent to the game server, which remains the one responsible for reacting to them and updating the game state accordingly. In the terminal, some event handlers are implemented to manage the terminal behaviour to them, such as the restart in case of the other player left, or the message to show after a game in the lobby menu.

In the end, the clients are responsible for handle the requests to create or join a game, which consists in posting the corresponding **LobbyEvent** to the central server.

3.3.2 Domain events

The domain events are divided into:

- **LobbyEvent**: events related to the **LobbyCoordinator**, which consumes them, while they are created and published by terminals and game servers:
 - **CREATE_GAME**: request to create a new game, published by a terminal;
 - **JOIN_GAME**: request to join an existing game, published by a terminal;
 - **DELETE_GAME**: request to delete an existing game, published by a game server;
 - **REQUEST_JOINABLE_GAMES**: request to get the list of joinable games, published by a terminal at its creation;
 - **COORDINATOR_STOPPED**: notification of the crash of a game server, published by a terminal to notify the lobby menu, in case it was still open.
- **InputEvent**: events related to the user inputs, which are sent by the clients to the game server to update the game state:

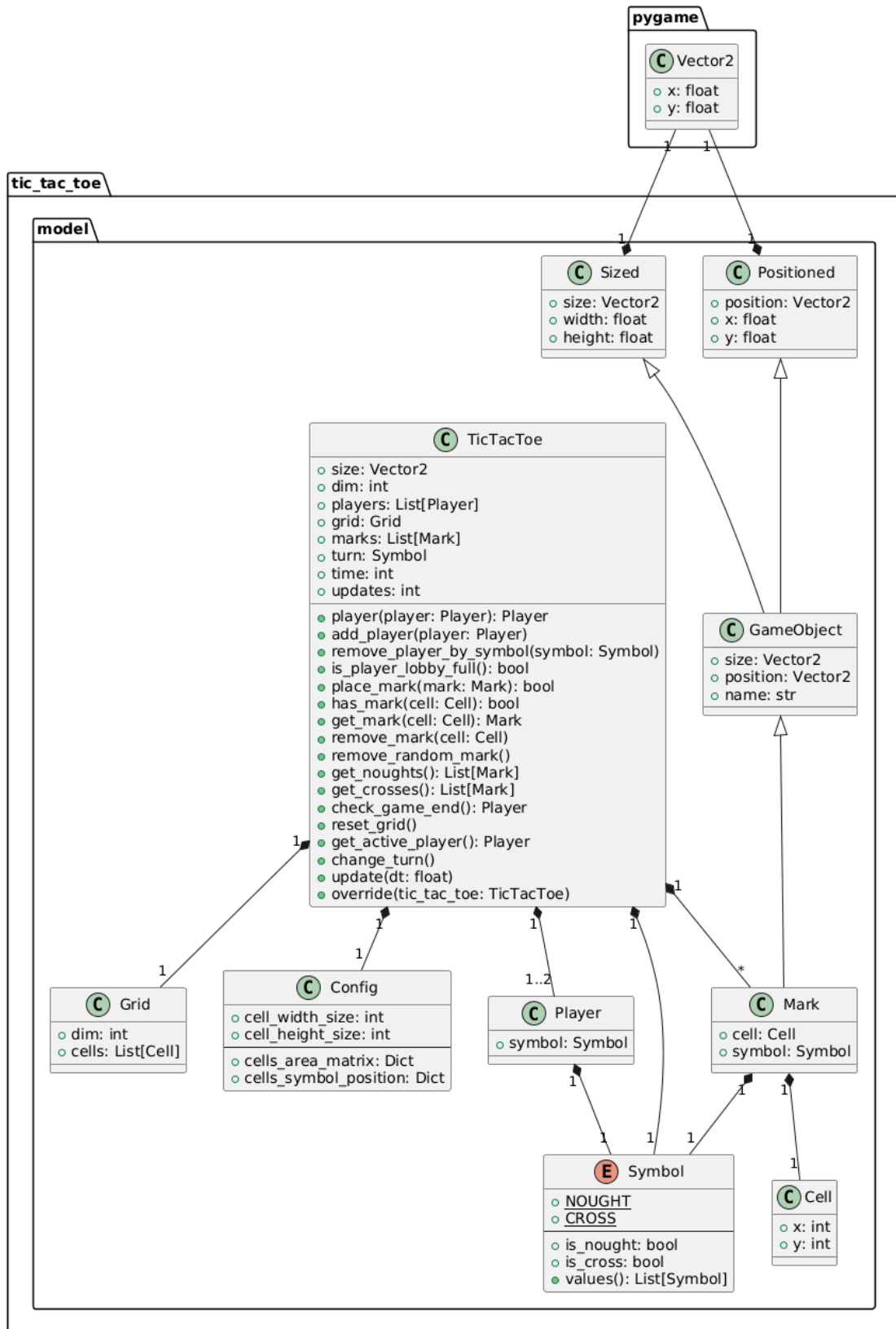


Figure 2: Class diagram of the domain model

- `MOUSEBUTTONDOWN`: mouse clicked on the GUI's grid to place a mark;
- `KEYDOWN`: `'Esc'` key pressed on the keyboard to leave the game.
- **ControlEvent**: *pygame* custom events related to the game state, which are exchanged between the game servers and the clients:
 - `PLAYER_CREATE_GAME`: request to create a new game triggered from the lobby menu and published by the corresponding terminal;
 - `PLAYER_JOIN_GAME`: request to join an existing game triggered from the lobby menu and published by the corresponding terminal;
 - `PLAYER_JOIN`: request of a player to join a game;
 - `PLAYER_LEAVE`: request of a player to leave a game;
 - `TIME_ELAPSED`: periodic event published by the game server to update the clients with the current game state or simply to notify the clients that the game is still running by sending the current elapsed time;
 - `GAME_OVER`: the game ended, which causes the quit from the game;
 - `MARK_PLACED`: a player has placed a mark on the grid;
 - `CHANGE_TURN`: switch the turn to the other player.

Events are handled by a controller, which is an `InputHandler` and an `EventHandler`. The class diagram in fig. 3 shows:

- the `InputHandler` that generates events of type `ControlEvent` and `LobbyEvent`, while processes events of type `InputEvent`, which are simply *pygame* events. It exploits an `ActionMap`, which is a mapping between key codes and `PlayerAction`. In addition, it generates the time passing events;
- an `EventHandler` that processes events of type `ControlEvent` and updates the game state accordingly;
- a `LobbyEventHandler` that processes events of type `LobbyEvent`.

3.3.3 Messages

The messages are all exchanged in *JSON* format, so they are serialized and deserialized with a specific implementation of a serializer and a deserializer. The messages exchanged are the events described in the section 3.3.2 and the text written in the client terminal to chat with the other player, as a string. The `TicTacToeTerminal` handles also messages sent by the `LobbyCoordinator`, formatted as dictionaries with the following keys:

- *joinable_games*: the list of joinable games;
- *coordinator*: the address of the game server to which the client has to connect;

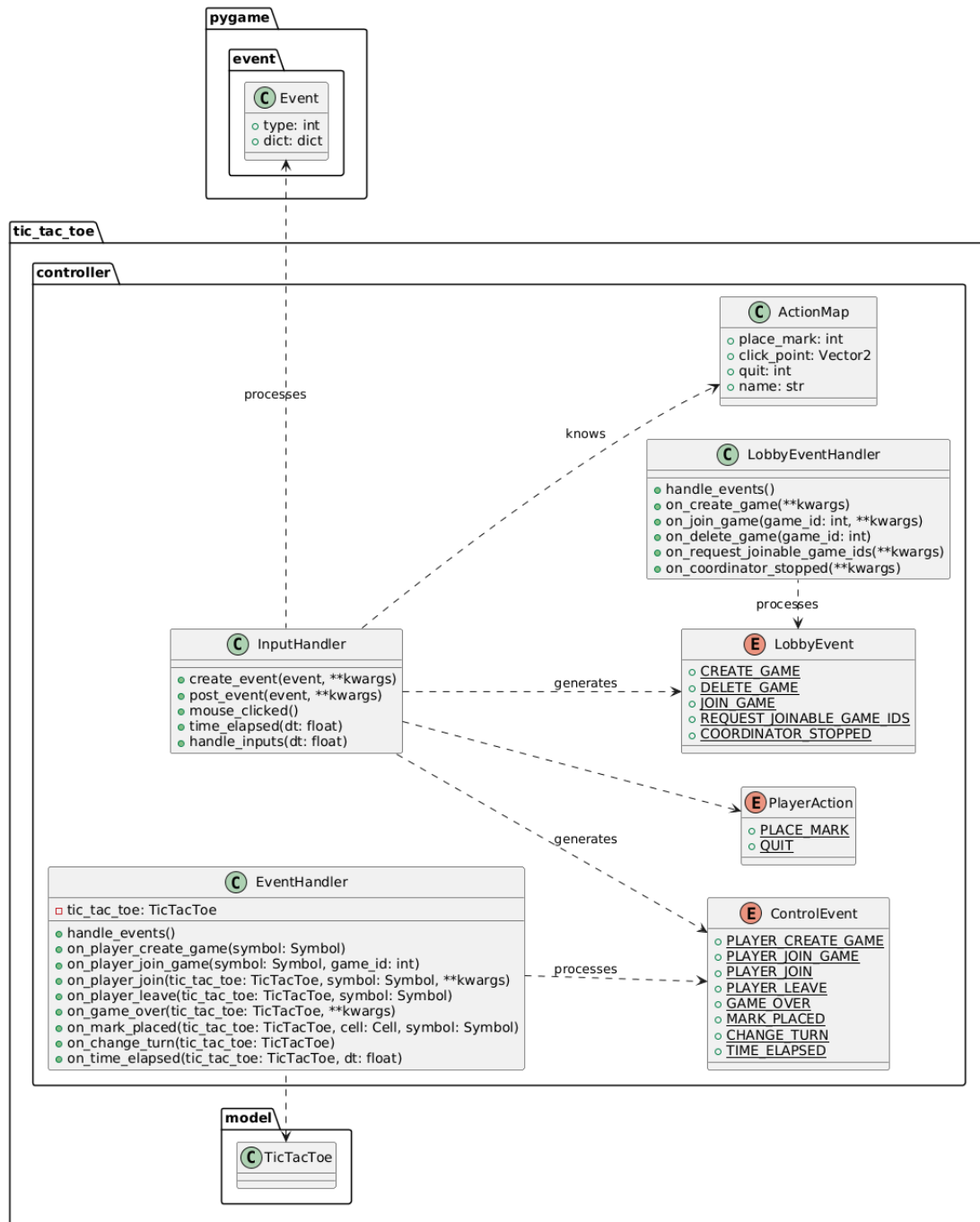


Figure 3: Class diagram of the controller

- *connection*: the address to which the message has to be sent;
- *error*: an error message to send.

The events exchanged between the clients and the servers can also be relative to the connection management:

- **ConnectionEvent**: events relative to a connection between a client and a server:
 - **MESSAGE**: a message is received from the other side of the connection;
 - **CLOSE**: close the connection;
 - **ERROR**: an error occurred during the connection.
- **ServerEvent**: events received by the servers:
 - **LISTEN**: the server is listening at the specified address;
 - **CONNECT**: open a connection with a client;
 - **STOP**: stop listening for new connections;
 - **ERROR**: an error occurred.

3.4 Interaction

The intra-components interactions are managed by means of events, exploiting the event queue of *pygame*, while the inter-components interactions are managed by means of messages exchanged through the network. For this reason, every component has its own event queue managed by the **EventHandler**.

3.4.1 Intra-component

The sequence diagram in fig. 4 shows the intra-component interaction, which is analogous for clients and servers. It consists of a loop in which the event handler checks for new events in the event queue and processes them one at a time by calling the corresponding handler method. The event queue is populated by the component itself, which publishes events by means of the input handler or because of a message received from the network.

In the following sections, only the inter-component interactions are shown.

3.4.2 Create game

The sequence diagram in fig. 5 shows that to start a new game, both the client and the central server have to be running. The **TicTacToeTerminal** starts the interaction by sending a **CREATE_GAME** event to the **LobbyCoordinator**. Therefore, the central server creates a new **TicTacToeCoordinator**, so the address of the new game server is sent back to the client, which can replace its connection to the central server with the one to the game server created. At this point, the client sends a **PLAYER_JOIN** event to the game server, which causes the game server to update the game state.

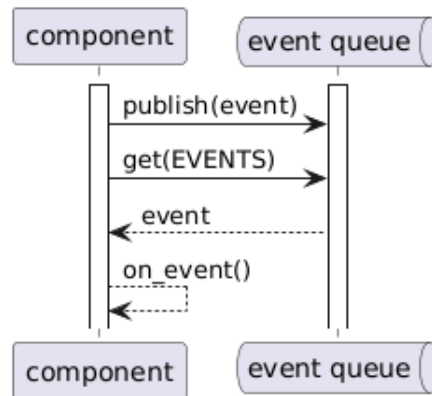


Figure 4: Sequence diagram of the intra-component interaction

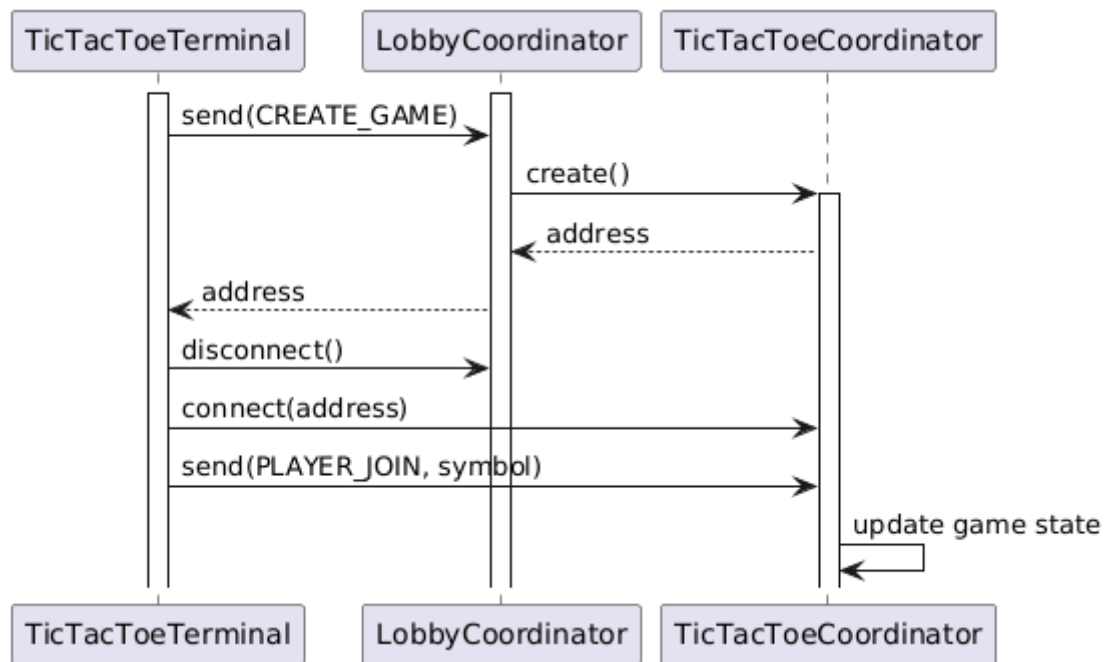


Figure 5: Sequence diagram of the create game interaction

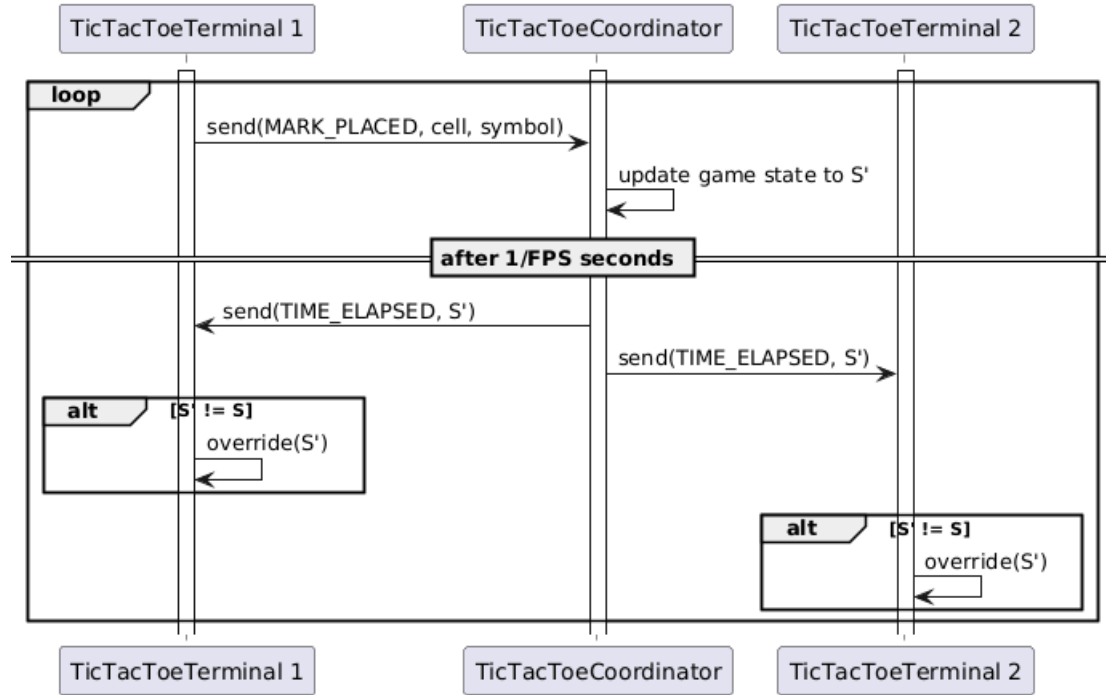


Figure 6: Sequence diagram of the update game state interaction

3.4.3 Update game state

In the sequence diagram pictured in fig. 6 is shown the continuous interaction between the clients and the game server during the game. The **TicTacToeTerminal** sends the **MARK_PLACED** event to the **TicTacToeCoordinator** when a player places a mark on the grid, so the game server updates its game state. In the second part of the diagram, it is shown how the game server sends every $1/\text{FPS}$ seconds the **TIME_ELAPSED** event to the clients, with the current game state included. When the clients receive the event, they override their game state with the one received from the server, only if they differ. In this way, the clients are synchronized with the game server and the game state is consistent between them.

3.4.4 Player leave

The sequence diagram in fig. 7 shows the scenario in which a player leaves the game by pressing the *Esc* key. The client is the one starting the interaction by sending the **PLAYER_LEAVE** event to the game server and restarting itself. The game server once received the event, propagates it to the other client, which analogously restarts itself. When the central server created the game server, it registered a callback in case of its termination, so due to the fact that the **TicTacToeCoordinator** treats the **PLAYER_LEAVE** and **GAME_OVER** events equivalently, it will terminate itself, causing the triggering of the

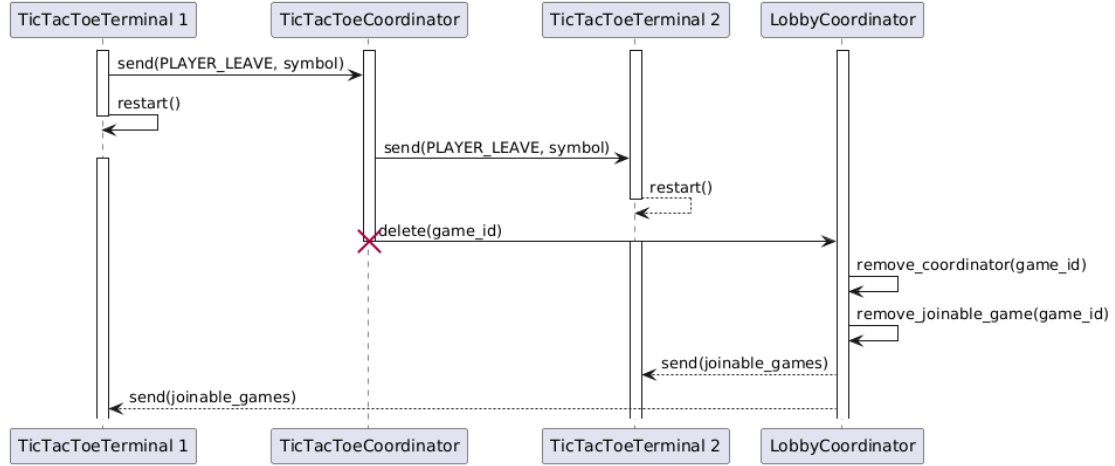


Figure 7: Sequence diagram of the player leave interaction

central server callback. The callback removes the game from the list of the joinable ones, in case the lobby was not full, and anyway it removes the coordinator from the list of the running ones.

Finally, all the clients subscribed to receive periodically the updated list of joinable games from the central server will receive the updated list.

3.4.5 Request joinable games

The sequence diagram in fig. 8 shows the interaction between a client and the central server, before the client starts its game loop, which consists of requesting the list of joinable games. The client as soon as it is created sends a **REQUEST_JOINABLE_GAMES** event to the central server and then it waits for the response with an upper bound timeout. The central server sends back to the client the list of joinable games and add it to the list of subscribed clients. The client at this point can start the lobby menu and the game loop.

During the entire lifecycle of the clients, until they are subscribed to the central server, they can receive the updated list of joinable games from it, which is sent every time a game is created, joined or deleted; in this way if the **TicTacToeTerminal** is still in the lobby menu, it can update the list of joinable games displayed accordingly.

3.4.6 Coordinator failure

The sequence diagram in fig. 9 shows the scenario in which the **LobbyCoordinator** crashes because of a failure. The central server crashing causes also the termination of all the game servers running. Therefore, all the clients that are executing inside their game loop, in particular the `handle_incoming_messages` method, will receive a **ConnectionResetError**, which causes their termination.

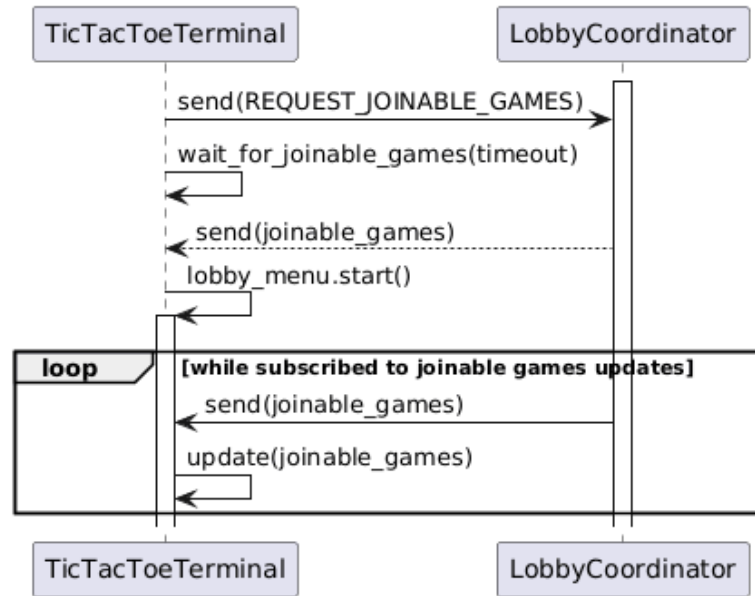


Figure 8: Sequence diagram of the request joinable games interaction

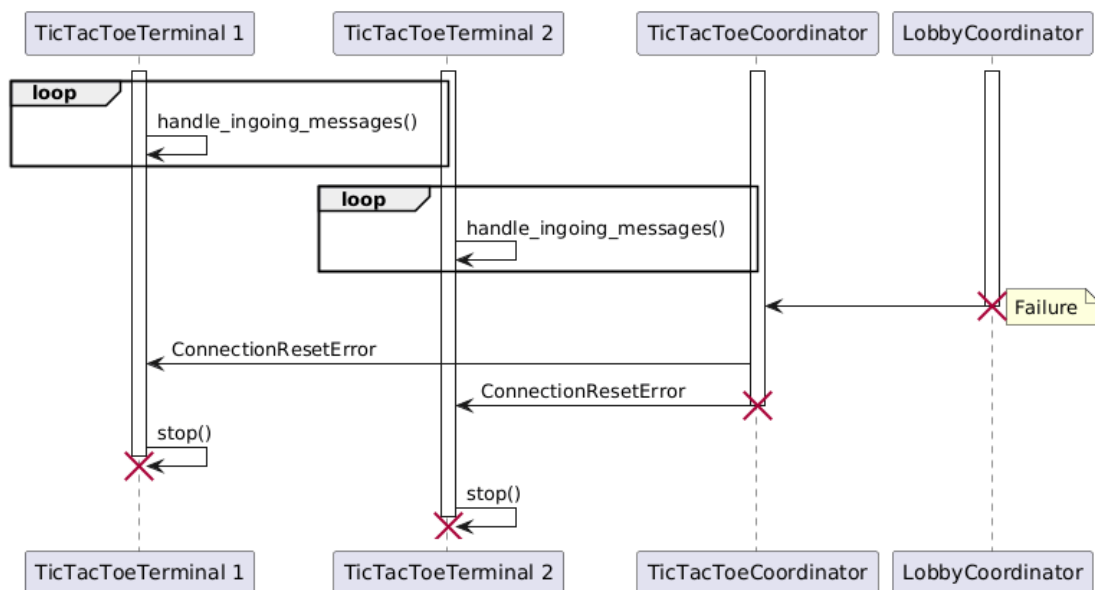


Figure 9: Sequence diagram of the coordinator failure interaction

3.5 Behaviour

The three main components are:

- **LobbyCoordinator**: as shown in the state diagram pictured in fig. 10, the central server is stateless, so it does not have any state to manage. Its only responsibility is to handle the requests from the clients and respond accordingly.
- **TicTacToeServer**: the state diagram in fig. 11 shows that the game server is stateful, in fact it is the authoritative source of the game state, so it is responsible for updating it and sending it periodically to the clients. Finally, it is possible to state that:
 - it listens for incoming inputs from the clients;
 - it executes the game loop, which consists of updating the game state and sending it to the clients;
- **TicTacToeTerminal**: the state diagram in fig. 12 emphasizes that the client is a stateful component, in fact it has its own copy of the game state, which is overridden by the one received from the game server, and then rendered by the view. In addition, it shows that the **TicTacToeTerminal** starts when it has received a response from the central server to its request to create or join a game made from the lobby menu, in order to connect to the game server. In the end, it is possible to state that:
 - it sends requests while in the lobby menu to the central server to create or join a game;
 - it sends the user inputs to the game server;
 - it listens for incoming game state updates from the game server;
 - it renders the game state on the GUI.

3.6 Data and Consistency Issues

No data is stored persistently, so there is no need for a database or any other storage component. The only information that needs to be stored is the list of the games that have been created and are joinable by another player, which is stored in a temporary *JSON* file on the central server. In addition, the list of game servers running is stored in memory on the central server. So every information stored is volatile and lost when the server is stopped.

The data about the game state is shared between the game server and the clients, so the consistency is guaranteed by the fact that the game server is the authoritative source of the game state, so the clients override theirs in case of differences.

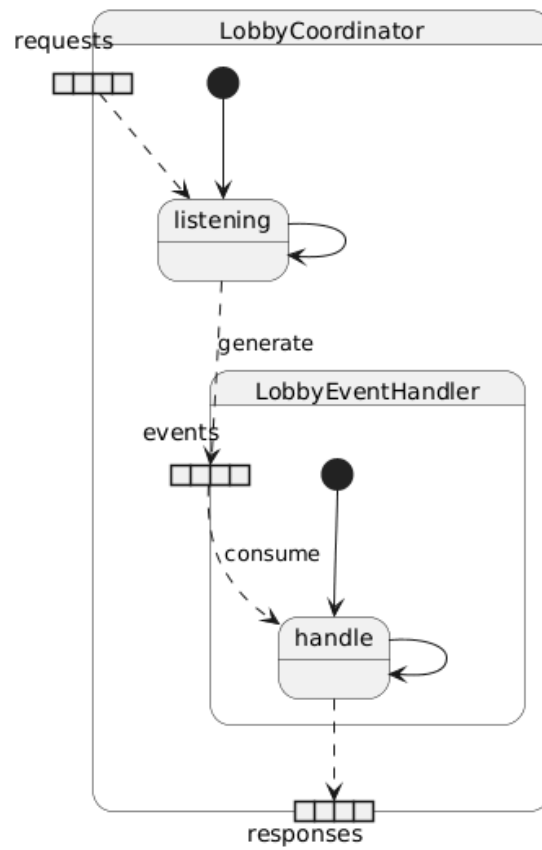


Figure 10: State diagram of the LobbyCoordinator

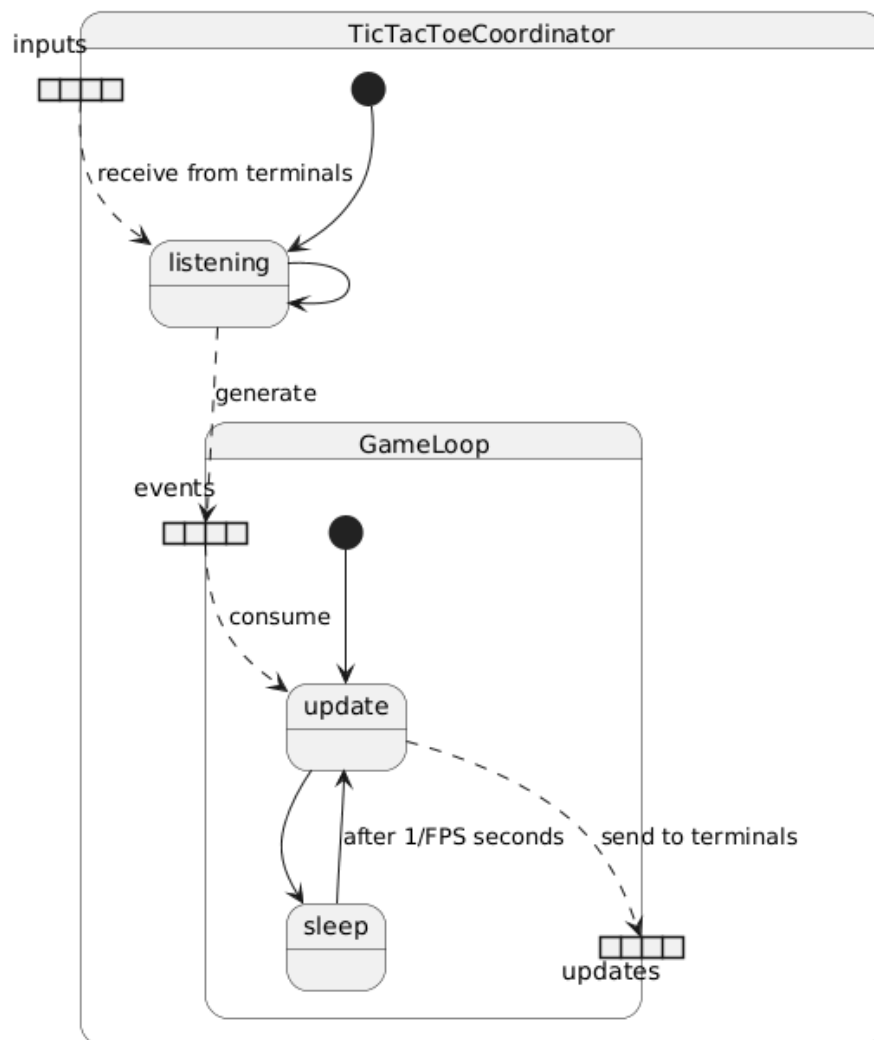


Figure 11: State diagram of the TicTacToeCoordinator

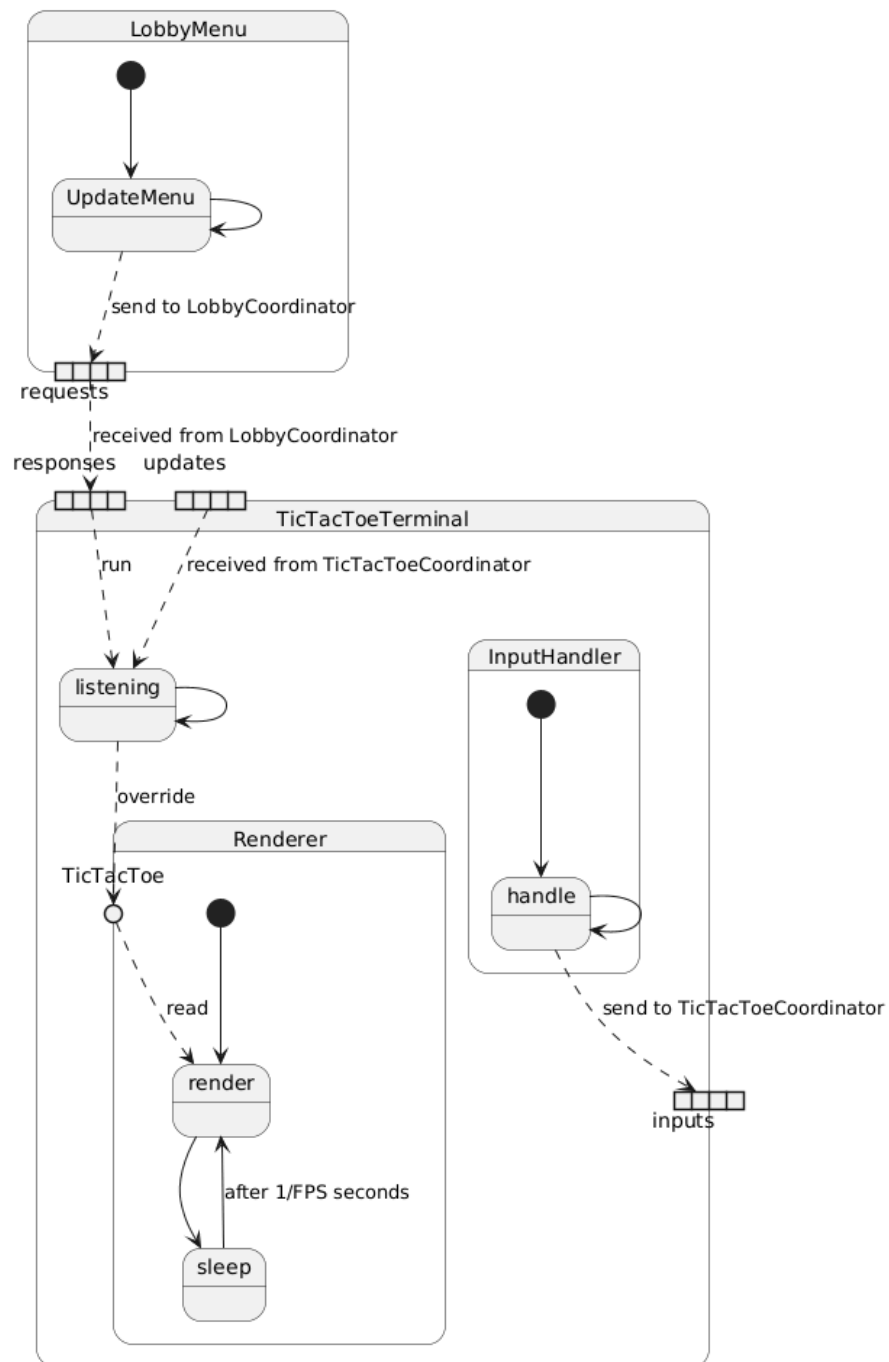


Figure 12: State diagram of the TicTacToeTerminal

3.7 Fault-Tolerance

The only form of replication is the one of the game state between the game server and the clients with its own game loop.

There is no heart-beating or retry mechanism, but when two components are connected, the one listening for incoming messages is continuously checking for new ones, so if the connection is lost, it will receive an error that can be handled accordingly.

The errors about component failures are handled as follows:

- in case of a game server failure, by terminating the clients connected to it and removing the game server from the list of the running ones in the central server;
- in case of a client failure, before being able to send the `PLAYER_LEAVE` event, the connection dies so the game server notices the crash, hence if the client was playing a game, the `GAME_OVER` event is posted and the game server terminated, so as the other clients playing the same game;
- in case of a central server failure, by terminating all the game servers and the clients connected to it or to a game server;
- in case a client starts when the central server is down, it simply terminates after more or less one second.

3.8 Availability

In case of network partitioning, following the *CAP theorem*, the system will prefer consistency over availability, so the clients will not be able to play the game until the connection with the game server is restored. This choice is based on the turn-based nature of the game and the not very high frequency of the interactions, which makes the consistency of the game state between the players more important than the availability of the game itself. Said that, in case of game server disconnection, the clients will be terminated and the game ended, like explained in section 3.7.

3.9 Security

The lack of persistent and sensitive data, other than the absence of a user management, makes no need for cryptographic schemas, and also no need for an authentication and authorization mechanism.

4 Implementation

- The network protocol chosen is TCP, in particular because of its reliability. The decision to not use UDP, even if it is the most common protocol for games and real time applications, is due to the turn-based nature of the game, which involves a low frequency of interactions, but an higher importance of the consistency of the

Listing 1: Central server message handling

```
1 def __handle_message(self, message: Any, **kwargs) -> None:
2     if any(e.matches(message) for e in (
3         LobbyEvent.CREATE_GAME,
4         LobbyEvent.JOIN_GAME,
5         LobbyEvent.REQUEST_JOINABLE_GAMES
6     )) and (CoordinationMessageType.CONNECTION.value in kwargs):
7         message.connection = kwargs[CoordinationMessageType.CONNECTION.value]
8         pygame.event.post(message)
```

game state between the players, so the package loss would be more critical than the latency.

- The in-transit data is represented in *JSON* format, with the following informations:
 - *type*: the type of the object;
 - *dict*: the dictionary composed by the name-value pairs of the object attributes.

Every object exchanged is serialized before being sent and deserialized when received, using the **Serializer** and **Deserializer** classes.

- As already said in section 3.6 the lack of persistent data does not require the use of a database, so the single temporary *JSON* file used to store the list of joinable games, as pairs of game's identifier and the symbol available that must be used to join the game, is queried by reading and deserializing it. In addition, as mentioned in section 3.9, there is no need for authentication and authorization mechanisms.
- In the listing 1 is shown the mechanism used by the central server to add to the message the TCP connection with the client, passed to the method as *kwargs*. In this way, the **LobbyCoordinator** can send back to the client the game server address to connect to, in case of creation or joining of a game request, alternatively the list of joinable games in case of that request.
- The mechanism used by the client to handle incoming messages is shown in listing 2, where in case of a message in dictionary format, the behaviour is determined by the key:
 - in case of error, the client terminates itself;
 - in case of the list of joinable games, it updates its local list, in order to update the lobby menu accordingly;
 - in case of the game server address, it replaces its connection to the central server with the one to the game server, so it can start the game loop.
- The game server handles the incoming messages from the clients according to the **ConnectionEvent**, as shown in listing 3:

Listing 2: Client message handling

```
1 def __handle_message(self, message: Any) -> None:
2     if isinstance(message, pygame.event.Event):
3         pygame.event.post(message)
4     elif isinstance(message, dict):
5         self.__handle_dict_message(message)
6     elif isinstance(message, str):
7         print(message)
8
9 def __handle_dict_message(self, message: Dict) -> None:
10    if CoordinationMessageType.ERROR.value in message:
11        self.logger.debug(message[CoordinationMessageType.ERROR.value])
12        self.stop()
13    elif CoordinationMessageType.JOINABLE_GAMES.value in message:
14        self.joinable_games = message[CoordinationMessageType.JOINABLE_GAMES.value]
15        self.joinable_games_updates.put(self.joinable_games)
16    elif CoordinationMessageType.COORDINATOR.value in message:
17        coord_address = Address(
18            message[CoordinationMessageType.COORDINATOR.value][0],
19            message[CoordinationMessageType.COORDINATOR.value][1]
20        )
21        self.logger.debug(f"Received coordinator address {coord_address}")
22        with self._lock:
23            old_client = self.client
24            old_client.close()
25            self.client = TcpClient(coord_address)
26            self.connected_to_coordinator = True
27            self.controller.post_event(ControlEvent.PLAYER_JOIN, symbol=self.symbol)
```

Listing 3: Server connection handling

```

1 def _broadcast_to_all_peers(self, message: Any) -> None:
2     event = serialize(message)
3     for peer in self.peers:
4         self.server.connections[peer].send(event)
5
6 def _on_message_received(self, event: ConnectionEvent, payload: str, connection:
7     TcpConnection, error: Exception) -> None:
8     match event:
9         case ConnectionEvent.MESSAGE:
10            if payload:
11                self.__handle_message(deserialize(payload), connection=connection)
12            case ConnectionEvent.CLOSE:
13                self.logger.debug(f"Connection with peer {connection.remote_address}
14                    closed")
15                self.remove_peer((connection.remote_address.host, connection.
16                    remote_address.port))
17                self.controller.post_event(ControlEvent.GAME_OVER)
18                self.tic_tac_toe.players.clear()
19            case ConnectionEvent.ERROR:
20                self.logger.debug(error)
21                self.remove_peer((connection.remote_address.host, connection.
22                    remote_address.port))
23
24 def __handle_message(self, message: Any, **kwargs) -> None:
25     if isinstance(message, pygame.event.Event):
26         if ControlEvent.PLAYER_JOIN.matches(message):
27             if CoordinationMessageType.CONNECTION.value in kwargs:
28                 message.connection = kwargs[CoordinationMessageType.CONNECTION.
29                     value]
30             elif ControlEvent.PLAYER_LEAVE.matches(message):
31                 self._broadcast_to_all_peers(message)
32             pygame.event.post(message)
33     elif isinstance(message, str):
34         self._broadcast_to_all_peers(message)
35         self.logger.debug(f"Received message: {message}")

```

- MESSAGE: the message is deserialized and passed to the `_handle_message` method with the TCP connection, in order to be broadcasted to every client by the `_broadcast_to_all_peers` method, either the chat message or the `PLAYER_LEAVE` event; for every other event the game server simply posts it and then react accordingly;
- CLOSE: the peer is removed and a `GAME_OVER` event is published;
- ERROR: the peer is removed.
- The chat between the players is implemented by means of a *daemon* thread in the client, in addition to the one for receiving messages from the game server. The listing 4 shows that the sender thread continuously get an item from the standard input queue and when an item is available, it sends it to the game server, which broadcasts it to the other client. The queue is supplied by the `StdinReader`, which runs a *daemon* thread to read the standard input and when a line is available, it puts it in the queue.

Listing 4: Chat implementation

```

1 class StdinReader:
2     """Single process-wide instance of a stdin reader."""
3
4     _instance = None
5     _instance_lock = threading.Lock()
6
7     def __init__(self):
8         self.queue: Queue[str] = Queue()
9         self._thread = threading.Thread(target=self._run, daemon=True)
10        self._thread.start()
11
12    def _run(self) -> None:
13        while True:
14            try:
15                line = input()
16            except (EOFError, KeyboardInterrupt):
17                self.stop()
18                return
19            self.queue.put(line)
20
21    @classmethod
22    def instance(cls) -> "StdinReader":
23        """Get the single instance of the stdin reader."""
24        with cls._instance_lock:
25            if cls._instance is None:
26                cls._instance = cls()
27            return cls._instance
28
29    def stop(self) -> None:
30        """Stop the stdin reader thread."""
31        self._thread.join(timeout=0.2)
32
33    def _send_message(self) -> None:
34        while self.running:
35            try:
36                msg = self._stdin_reader.queue.get(timeout=0.2)
37            except Empty:
38                continue
39            if self.running:
40                try:
41                    self.logger.debug(f"Send {msg} to the opponent")
42                    self.client.send(serialize(self.message(msg, f"Player '{self.
43                        symbol.value}'")))
44                except Exception as e:
45                    self.logger.debug(f"Could not send message, connection closed: {e}
46                        ")
47                    break

```

4.1 Technological details

The project is implemented in *Python* and it exploits the following technologies:

- *pygame* for the GUI and the event management;
- *pygame-menu* for the lobby menu GUI;
- *socket* for the network communication;
- *json* for the serialization and deserialization of the messages;
- *threading* for the concurrent execution of the game loop and the listening for incoming messages, and in the case of clients, also for the chat message sending;
- *multiprocessing* for the concurrent execution of the game servers and the central server;
- *argparse* for the command line arguments parsing.

5 Validation

5.1 Automatic Testing

Individual components are unit-tested and the tests can be run using **Poetry**, so first of all execute:

```
1 | poetry install
```

Unit tests are divided into two groups:

- *model*: where the domain model is tested, for example the game state, game objects, symbols, marks, etc. The game state is tested by verifying the correct update after the execution of the main actions, including its override, which is used frequently by the clients to synchronize their game state with the one of the game server. It is possible to run the tests with the following command:

```
1 | poetry run test-model
```

- *presentation and communication*: where the serialization and deserialization of objects are tested, other than the TCP communication between clients and servers. In particular, the communication tests focus on the connection between a client and a server, and also the correctness of the messages exchanged between them. In addition, the correctness of a game session is verified by testing an instance of `TicTacToeGame`, where both the game loop and the creation of controller and view are tested. To run the tests, execute the command:

```
1 | poetry run test
```

5.2 Acceptance test

Manual testing was performed for:

- input handling, which is easier to test manually, by interacting with the GUI and verifying the correct update of the game state;
- view rendering and lobby menu;
- the game state synchronization between the clients and the game server, even though the game state override is also unit-tested automatically, its update and synchronization with the central server one is strictly related to the game loop and the event handling;
- the possibility to play multiple games simultaneously, without conflicts between them, by running multiple clients and a single central server, which creates multiple game servers, one for each game, in a single machine;
- the chat between the players, which is easier to test manually to verify its correctness during a game session.

6 Deployment

1. Starting from the root of the project, it is possible to run the system by means of **Poetry**, so first of all install it;
2. install the dependencies of the project by executing on a terminal:

```
poetry install
```

3. run the central server by executing on a terminal:

```
poetry run lobby-coordinator
```

4. run the clients by executing on different terminals:

```
poetry run terminal
```

The lobby menu interface will be displayed and from there it is possible to create or join a game.

In alternative, it is possible to run the system directly from the CLI, specifying some arguments, which can be listed by executing:

```
poetry run python -m tic_tac_toe --help
```

The arguments can be added both to the commands to run the central server and the clients, which are respectively:

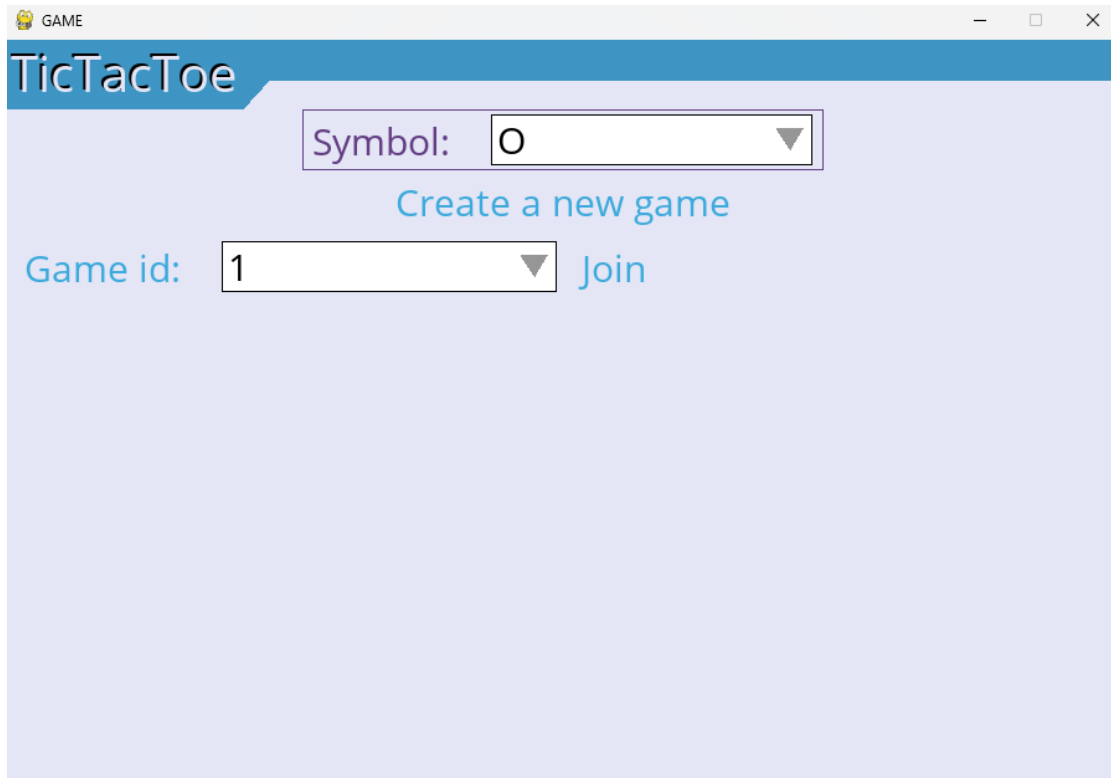


Figure 13: Lobby menu with a game already joinable

```
poetry run python -m tic_tac_toe -m centralised -r coordinator
```

```
poetry run python -m tic_tac_toe -m centralised -r terminal
```

7 User Guide

Considering the central server is already running, the user can start a client and the first thing that will be shown is the lobby menu, pictured in fig. 13, where it is always possible to create a new game, choosing the symbol to play with, while the possibility to join a game is available only if there is at least one joinable game. When a game id is selected from the joinable games list, the symbol selector will show only the symbol available.

After creating a game, the player will see the waiting screen, pictured in fig. 14, and then when another player joins the game, the game will start and the waiting screen will be replaced by the game interface, pictured in fig. 15, which is instead immediately displayed when the player joins an existing game.

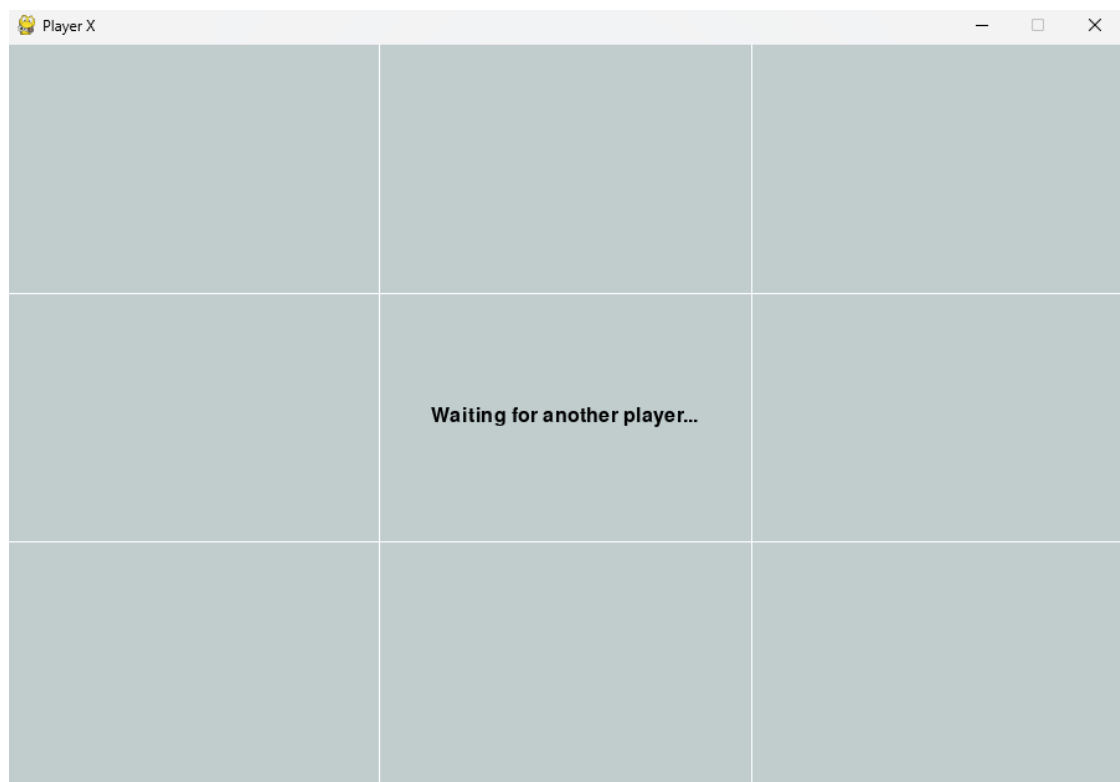


Figure 14: Waiting screen

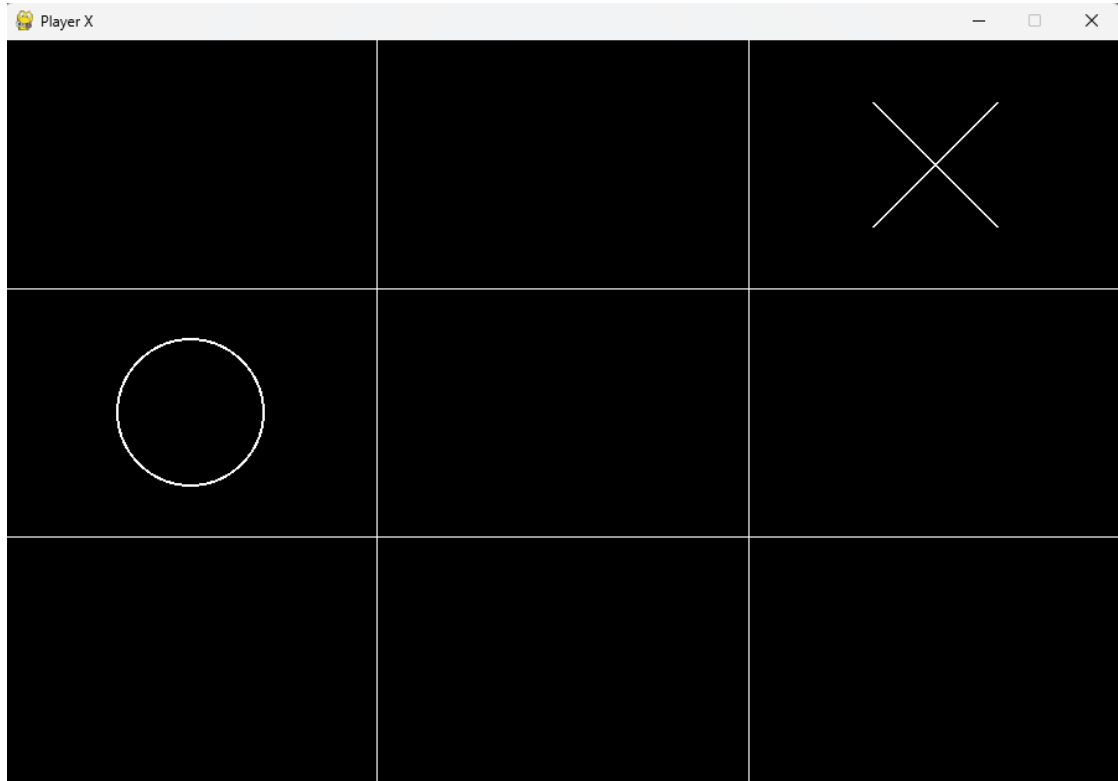


Figure 15: Game interface

In the game interface, it is displayed the state of the game in the grid, and the active player can place a mark in an empty cell by clicking on it with the left mouse button.

In any moment of the game, the player can leave it by pressing the *Esc* key, which will bring all the players of the game back to the lobby menu.

In addition, the players can chat with each other by typing a message in the CLI and pressing *Enter* to send it, also the messages received by the other player will be displayed there.

Finally, every message that the game server or the central server wants the client to display, will be printed in the CLI.

8 Future works

There are several possible improvements and extensions that could be made to enhance the playability of the game. First of all, the fault-tolerance and scalability of the system could be improved by implementing a replication mechanism, so that in case of a game server or central server failure, the players could continue to play the game; otherwise, a recovery mechanism could be implemented, in order to restore the game state after a crash.

Another possible addition could be a user management system, which would allow players to create accounts and log in, so an history of past games could be stored.

Finally, all the user interfaces could be improved, meaning their design and usability, in order to make the game more appealing and also increase the clarity of the user experience.